Unit-3

1

How to declare File pointer, Open a File with various modes, Close a file with suitable example.

To declare a file pointer in C, we use the following syntax:

FILE *fp;

Here, FILE is the typedef name of the predefined file pointer structure and fp is a pointer variable of type FILE.

To open a file with various modes, we use the fopen() function. The syntax for opening a file is as follows:

FILE *fopen(const char *filename, const char *mode);

Here, filename is the name of the file to be opened, and mode specifies the mode in which the file is to be opened. The following table lists the different modes in which a file can be opened:

Table

Mode	Description
r	Opens a file for reading. The file must exist.
w	Opens a file for writing. If the file exists, its contents are truncated. If the file does not exist, it is created.
a	Opens a file for appending. If the file exists, the data is written at the end of the file. If the file does not exist, it is created.

2 List and discuss file opening modes in C.

In C, you can open files in different modes depending on the purpose of opening the file. Here are the four major file opening modes:

1.Read mode ®: This mode is used to open an existing file for reading. If the file does not exist, the fopen() function returns NULL [1].

2.Write mode (w): This mode is used to create a new file or overwrite an existing file. If the file exists, its contents are overwritten. If the file does not exist, it will be created [1].

3.Append mode (a): This mode is used to open an existing file for writing at the end of the file. If the file does not exist, it will be created .

4.Read and write mode (r+): This mode is used to open an existing file for both reading and writing. If the file does not exist, the fopen() function returns NULL .

3 Explain File functions for Reading and Writing data from files

In C, there are several functions that can be used to read and write data from files. Here are some of the most commonly used file functions:

1.fopen(): This function is used to open a file. It takes two arguments: the name of the file and the mode in which the file is to be opened. The mode can be “r” for reading, “w” for writing, “a” for appending, or “r+” for both reading and writing [1].

2.fclose(): This function is used to close a file that has been opened using fopen(). It takes one argument: the file pointer returned by fopen() [1].

3.fread(): This function is used to read data from a file. It takes four arguments: a pointer to the buffer where the data is to be stored, the size of each element to be read, the number of elements to be read, and the file pointer returned by fopen() .

4.fwrite(): This function is used to write data to a file. It takes four arguments: a pointer to the buffer containing the data to be written, the size of each element to be written, the number of elements to be written, and the file pointer returned by fopen() [1].

5.fseek(): This function is used to move the file pointer to a specific location in the file. It takes three arguments: the file pointer returned by fopen(), the number of bytes to move the pointer, and the origin from which to move the pointer (the beginning of the file, the current position, or the end of the file) [1].

6.ftell(): This function is used to determine the current position of the file pointer. It takes one argument: the file pointer returned by fopen() [1].

7.fprintf(): This function is used to write formatted data to a file. It takes two arguments: the file pointer returned by fopen() and a format string that specifies the format of the data to be written [1].

8.fscanf(): This function is used to read formatted data from a file. It takes two arguments: the file pointer returned by fopen() and a format string that specifies the format of the data to be read [1].

4 What is the use of binary file with example. Compare binary file with text file.

A binary file is a file that stores data in a non-textual format, such as images, audio, video, executable code, compressed data, etc. A binary file can only be read or written by a program that knows how to interpret the binary data. A binary file is usually more compact and efficient than a text file, as it does not need to encode the data using characters. A binary file can also store any type of data, regardless of its structure or meaning.

A text file is a file that stores data in a textual format, such as letters, numbers, symbols, etc. A text file can be read or written by any program that can handle text, such as a text editor, a word processor, or a web browser. A text file is usually more readable and editable than a binary file, as it uses characters that are human-understandable. A text file can only store data that can be represented by characters, such as plain text, source code, HTML, etc.

The main differences between binary files and text files are:

- Binary files store data in a non-textual format, while text files store data in a textual format.
- Binary files can only be read or written by programs that know how to interpret the binary data, while text files can be read or written by any program that can handle text.
- Binary files are usually more compact and efficient than text files, while text files are usually more readable and editable than binary files.
- Binary files can store any type of data, regardless of its structure or meaning, while text files can only store data that can be represented by characters.

5 Explain functions for random access of file (ftell and fseek) with suitable example.

The functions for random access of file in C are:

- `ftell()`: This function returns the current position of the file pointer in a random access file. It takes a file pointer as an argument and returns a long integer value that represents the number of bytes from the beginning of the file. For example:

```
FILE *fp = fopen("test.txt", "r");
long pos = ftell(fp); // returns 0 if the file is opened successfully
```

- `fseek()`: This function moves the file pointer to a specified position in a random access file. It takes three arguments: a file pointer, an offset value, and a mode. The offset value is the number of bytes to move the file pointer from a reference point, which is determined by the mode. The mode can be one of the following constants:
  - `SEEK_SET`: The reference point is the beginning of the file.
  - `SEEK_CUR`: The reference point is the current position of the file pointer.
  - `SEEK_END`: The reference point is the end of the file.

  The function returns 0 if the operation is successful, or -1 if an error occurs. For example:

```
FILE *fp = fopen("test.txt", "r");
fseek(fp, 10, SEEK_SET); // moves the file pointer 10 bytes from the beginning of the file
fseek(fp, -5, SEEK_CUR); // moves the file pointer 5 bytes backward from the current position
fseek(fp, 0, SEEK_END); // moves the file pointer to the end of the file
```

- `rewind()`: This function moves the file pointer to the beginning of a random access file. It is equivalent to `fseek(fp, 0, SEEK_SET)`. It takes a file pointer as an argument and does not return any value. For example:

```
FILE *fp = fopen("test.txt", "r");
rewind(fp); // moves the file pointer to the beginning of the file
```

6 Explain how to remove and rename a file with example.

To remove and rename a file in C, we can use the standard library functions `remove` and `rename`, which are defined in the `<stdio.h>` header file. These functions take the file names as arguments and return 0 on success or -1 on failure. For example:

```
#include <stdio.h>
int main()
{
    char filename[] = "test.txt";
    if (remove(filename) == 0)
    {
        printf("File %s removed successfully.\n", filename);
    }
    else
    {
        perror("Error removing file");
    }
    char oldname[] = "old.txt";
    char newname[] = "new.txt";
    if (rename(oldname, newname) == 0)
    {
        printf("File %s renamed to %s.\n", oldname, newname);
    }
    else
    {
        perror("Error renaming file");
    }
    return 0;
}
```

This program tries to remove a file named `test.txt` and rename a file named `old.txt` to `new.txt`. It prints a success message if the operations are successful, or an error message if they fail.

7 Explain Command line Arguments with suitable example

Command line arguments are the values that are given after the name of the program in the command line shell of the operating system. They are used to pass some information or data to the program when it is executed. For example, if we have a program named `sum.c` that calculates the sum of two numbers, we can pass the numbers as command line arguments, such as:

```
$ gcc sum.c -o sum # compile the program
$ ./sum 10 20 # run the program with arguments 10 and 20
The sum is 30
```

To use command line arguments in C, we need to define the `main` function with two parameters: `int argc` and `char *argv[]`.
The `argc` parameter is the number of command line arguments, including the name of the program. The `argv` parameter is an array of strings that store the command line arguments. The first element of the array, `argv[0]`, is the name of the program. The rest of the elements, from `argv[1]` to `argv[argc-1]`, are the command line arguments. For example, the `sum.c` program can be written as:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
  // check if the number of arguments is valid
  if (argc != 3)
  {
    printf("Usage:  %s num1 num2\n", argv[0]);
    return 1;
  }
```

```
// convert the arguments to integers
  int num1 = atoi(argv[1]);
  int num2 = atoi(argv[2]);

  // calculate and print the sum
  int sum = num1 + num2;
  printf("The sum is %d\n", sum);

  return 0;
}
```

Command line arguments are useful for controlling the behavior of the program from outside, instead of hard-coding the values inside the code.

Unit -4 Bitwise Operators and Pre-processor Directives

1 What do you mean by Bitwise Operators. Explain each bitwise operators with suitable example.

Bitwise operators are operators that perform operations on the individual bits of a binary number. They are useful for manipulating and testing bits, such as setting, clearing, toggling, or checking bits. There are six bitwise operators in C:

- & (bitwise AND): This operator performs a logical AND operation on each pair of corresponding bits of two operands. The result is 1 if both bits are 1, otherwise 0. For example:

```
int a = 12; // 00001100 in binary
int b = 25; // 00011001 in binary
int c = a & b; // 00001000 in binary, 8 in decimal
```

- | (bitwise OR): This operator performs a logical OR operation on each pair of corresponding bits of two operands. The result is 1 if either bit is 1, otherwise 0. For example:

```
int a = 12; // 00001100 in binary
int b = 25; // 00011001 in binary
int c = a | b; // 00011101 in binary, 29 in decimal
```

- ^ (bitwise XOR): This operator performs a logical XOR (exclusive OR) operation on each pair of corresponding bits of two operands. The result is 1 if the bits are different, otherwise 0. For example:

```
int a = 12; // 00001100 in binary
int b = 25; // 00011001 in binary
int c = a ^ b; // 00010101 in binary, 21 in decimal
```

- ~ (bitwise NOT): This operator performs a logical NOT operation on each bit of a single operand. The result is the complement of the operand, i.e., 0 becomes 1 and 1 becomes 0. For example:

```
int a = 12; // 00001100 in binary
int b = ~a; // 11110011 in binary, -13 in decimal
```

- << (left shift): This operator shifts the bits of the left operand to the left by the number of positions specified by the right operand. The vacated bits on the right are filled with zeros. The result is equivalent to multiplying the left operand by 2 raised to the power of the right operand. For example:

```
int a = 12; // 00001100 in binary
int b = a << 2; // 00110000 in binary, 48 in decimal
```

- >> (right shift): This operator shifts the bits of the left operand to the right by the number of positions specified by the right operand. The vacated bits on the left are filled with zeros or the sign bit, depending on whether the left operand is unsigned or signed. The result is equivalent to dividing the left operand by 2 raised to the power of the right operand. For example:

```
int a = 12; // 00001100 in binary
int b = a >> 2; // 00000011 in binary, 3 in decimal
```

2 Discuss Conditional directives in brief with example

Conditional directives are preprocessor directives that allow us to include or exclude parts of a program based on certain conditions. They are useful for controlling the compilation process, such as testing different versions of code, debugging, or defining platform-specific features. There are four main conditional directives in C:

- `#if`: This directive evaluates a constant expression and checks if it is non-zero. If it is, the following block of code is compiled, otherwise it is skipped. For example:

```
#if DEBUG
  printf("Debug mode is on\n");
#endif
```

This code will only be compiled if the macro DEBUG is defined and has a non-zero value.

- `#else`: This directive is used with `#if` to provide an alternative block of code to be compiled if the `#if` condition is false. For example:

```
#if DEBUG
  printf("Debug mode is on\n");
#else
  printf("Debug mode is off\n");
#endif
```

This code will print “Debug mode is on” if the macro DEBUG is defined and has a non-zero value, otherwise it will print “Debug mode is off”.

- `#elif`: This directive is used with `#if` to provide multiple conditions to be checked in sequence. It is equivalent to “else if” in C. For example:

```
#if OS == 1
  printf("Windows\n");
#elif OS == 2
  printf("Linux\n");
#elif OS == 3
  printf("MacOS\n");
#else
  printf("Unknown\n");
#endif
```

This code will print the name of the operating system based on the value of the macro OS.

- `#endif`: This directive marks the end of a conditional block. It must be used with every `#if`, `#else`, or `#elif` directive. For example:

```
#if DEBUG
  printf("Debug mode is on\n");
#endif
```

This code ends the conditional block started by `#if`.

3 Explain Unconditional directives with suitable example.

Unconditional directives are preprocessor directives that do not depend on any condition and are always executed by the preprocessor. They are used to perform various tasks such as including files, defining macros, undefining macros, etc. There are three main unconditional directives in C:

- `#include`: This directive is used to include the contents of another file in the source code. It can be used to include header files that contain function prototypes, macro definitions, or other declarations. There are two ways to use this directive:
  - `#include <file.h>`: This includes a file from the standard library or the system directory. For example:

```
#include <stdio.h> // includes the standard input/output header file
```

  - `#include "file.h"`: This includes a file from the current directory or the user-specified path. For example:

```
#include "myheader.h" // includes a user-defined header file
```

- `#define`: This directive is used to define a macro, which is a name for a constant value, an expression, or a piece of code. The preprocessor replaces every occurrence of the macro name with its value or code before compilation. For example:

```
#define PI 3.14 // defines a macro for the value of pi
#define SQUARE(x) (x * x) // defines a macro for the square of a number
#define DEBUG // defines an empty macro for conditional compilation
```

  - `#undef`: This directive is used to undefine a macro, which means to remove its definition and stop its replacement by the preprocessor. It can be used to avoid conflicts or errors with other macros or identifiers. For example:

```
#undef PI // undefines the macro for the value of pi
#undef SQUARE // undefines the macro for the square of a number
#undef DEBUG // undefines the macro for conditional compilation
```

4 Differences:

Write the difference between Macro & Function.

Difference between Macro and Function

Macro	Function
It is not compiled, it is pre-processed.	It is not pre-processed but it is compiled.
Macros do not check for compilation error which leads to unexpected output.	Function checks for compilation error and there is a less chance of unexpected output.
Code length is increased.	Code length remains same.
Macros are faster in execution than function.	Functions are bit slower in execution.
Before compilation process the macro name is replaced by the macro value.	In a function call, transfer of control takes place.
Macros are useful when a small piece of code used multiple times in a program.	Functions are helpful when a large piece of code repeated number of times.

What is the difference between #include”stdio.h” and #include<stdio.h>

The difference between #include”stdio.h” and #include<stdio.h> is in the location where the preprocessor searches for the header file to be included in the code. #include<stdio.h> is used for standard library headers, such as stdio.h, which are located in the system directories. #include”stdio.h” is used for user-defined headers, such as myheader.h, which are located in the current directory or the user-specified path. For example:

```
#include <stdio.h> // includes the standard input/output header file from
the system directory

#include "myheader.h" // includes the user-defined header file from the
current directory or the user-specified path
```

5 Write a short notes on #pragma directive in C.

The #pragma directive in C is a special purpose directive that is used to provide additional information or instructions to the compiler, beyond what is conveyed in the language itself. The #pragma directive is compiler-specific, which means that the behavior and meaning of the directive may vary from one compiler to another. The forms of the directive specified by the C standard are prefixed with STDC, such as `#pragma STDC FENV_ACCESS`. A C compiler is free to attach any meaning it likes to other pragmas, such as `#pragma pack` or `#pragma once`.
Some of the common uses of the #pragma directive in C are:

- To control the alignment of structure members, such as `#pragma pack (n)`, where n is the number of bytes for alignment.
- To specify the functions that are executed before the program starts or after the program exits, such as `#pragma startup` and `#pragma exit`.
- To suppress or enable certain warnings or errors, such as `#pragma warn` or `#pragma error`.
- To include or exclude some parts of the code based on some conditions, such as `#pragma ifdef` or `#pragma ifndef`.
- To optimize the performance or memory usage of the code, such as `#pragma optimize` or `#pragma loop`.
- To mark some sections of the code as parallelizable, such as `#pragma omp` for OpenMP.